

Exercise 1 - BlackJack (CLI)

General Guidelines

- Questions should be sent via [email](#).
- Submission deadline: June 23rd.
- Program must validate input and exit correctly when needed.
- Do not print unnecessary or irrelevant output.
- Program must run with no errors or warnings.
- **This is an individual assignment.**

Technical Guidelines

- Use random() for both user and CPU card drawing.
- Cards are values 1-13 (11=Jack, 12=Queen, 13=King).
- Pay attention to the "Ace" card which can act either as 1 or 11. In case the sum is greater than 21 and one of his cards is 11 (jack), it turns to 1 automatically and user can continue draws based on his judgment.
- Ignore real deck probability (the fact that a single deck contains 52 cards and each number is represented 4 times can be ignored).

Exercise Description

Develop a **BlackJack** CLI game in Python. The game starts by printing the welcome art from blackjack_art.py (attached) and proceeds with a turn-based interaction between the user and CPU.

Detailed Game Flow

User Scenario (starts first)

1. Display welcome art.
2. Draw the first card for the user and display it.
3. Ask the user if he wants another card (yes/no).
4. If yes: draw card, update sum, display sum and all his cards.
5. Repeat 3 & 4 until the user stops or exceeds 21.
6. If sum > 21: print 'You Lost' and exit.
7. If sum < 21, move the CPU.

CPU Scenario

1. CPU starts after the user stops drawing.
2. CPU draws cards while its sum is less than the user's sum.

3. After each draw, display CPU action and updated sum.
4. If CPU sum > user sum and ≤ 21 : print 'CPU won' and exit.
5. If both reach 21: print 'It's a tie'.

Submission Requirements

- Submit Python source files.
- Program must run from CLI.
- Code must be clean and readable.

Grading Criteria

- Correctness. (50 pt)
- Logic. (20 pt)
- Style/PEP 8. (10 pt)
- Documentation. (5 pt)
- Efficiency. (15 pt)